Gestão de Projecto de Software

Licenciatura em Engenharia Informática

2024/2025

Modelos de desenvolvimento de software
Métodos Ágeis
SCRUM (1)

A3

86

Métodos ágeis, porquê ? Qual a resposta que os métodos ágeis podem dar ?(1)

- Flexibilidade e Adaptabilidade
  - as equipas de desenvolvimento têm que ter processos flexíveis e adaptáveis às mudanças.
    - Em vez de seguir um plano rígido, como no modelo tradicional de desenvolvimento em cascata, os métodos ágeis facilitam ajustes contínuos com base no feedback do cliente e nas mudanças de requisitos.
    - Há ambientes de negócios onde as necessidades e as prioridades podem mudar, ou evoluírem, rapidamente.
- Entrega Rápida de Valor
  - entregas frequentes e incrementais de software funcional, permite que as empresas forneçam valor aos clientes mais rapidamente.
    - Em vez de esperar meses ou anos por uma grande versão, o desenvolvimento ágil possibilita lançamentos em ciclos curtos, muitas vezes em semanas.
    - especialmente importante em mercados competitivos, onde a capacidade de lançar rapidamente novas funcionalidades pode ser um fator diferenciador significativo.

87

Métodos ágeis, porquê ? Qual a resposta que os métodos ágeis podem dar ?(2)

- Foco no Cliente
  - foco constante nas necessidades e no feedback dos clientes.
  - reuniões regulares com as partes interessadas e revisões frequentes do trabalho em andamento, as equipas podem garantir que o produto final realmente atenda às expectativas e necessidades dos utilizadores finais.
  - reduzir o risco de construir um produto que não seja útil ou que esteja desactualizado.
- Melhoria Contínua
  - incorporar práticas de retrospectiva, onde as equipas analisam regularmente o que funcionou bem e o que pode ser melhorado.
  - Este processo de aprendizagem contínua permite que as equipas aprimorem constantemente as suas práticas de trabalho, aumentando a eficiência e a eficácia ao longo do tempo.

88

88

Métodos ágeis, porquê ? Qual a resposta que os métodos ágeis podem dar ?(3)

- Redução de Riscos
  - Ao dividir o desenvolvimento em pequenos incrementos e realização de testes contínuos permite identificar e corrigir problemas mais cedo no processo de desenvolvimento.
    - reduz significativamente os riscos de grandes falhas ou defeitos no software, que podem ser muito mais caros e demorados para corrigir em fases posteriores do desenvolvimento.
- Motivação e Envolvimento da Equipa
  - Promoção de uma cultura de colaboração, autonomia e responsabilidade entre os membros da equipa.
    - maior envolvimento nas decisões e um ambiente de trabalho mais dinâmico, os membros da equipa geralmente sentem-se mais motivados e envolvidos.
    - Esta motivação é essencial para manter a produtividade e a satisfação no trabalho, especialmente em projetos complexos de longo prazo.

89

89

Métodos ágeis, porquê ? Qual a resposta que os métodos ágeis podem dar ?(4)

- Melhoria na Qualidade do Software
  - a prática de testes contínuos e integração contínua, os métodos ágeis permitem que as equipas identifiquem problemas de qualidade de forma antecipada e frequente.
  - software de melhor qualidade, com menos bugs e mais alinhado às expectativas dos utilizadores finais.

90

90

Como e quando surgem as metodologias ágeis

- Em Fevereiro de 2001, na estância de esqui The Lodge at Snow-bird, nas montanhas do Utah – EUA
  - 17 pessoas reúnem-se para esquiar, relaxar e tentar encontrar princípios e valores comuns na área do desenvolvimento de software usando metodologias ágeis
  - Todos usavam metodologias ágeis em vez dos métodos tradicionais (maior enfase na documentação e no planeamento a longo prazo)
  - **O documento resultante da reunião designou-se por Manifesto Ágil**

91

91

Manifesto for Agile Software Development

We are uncovering better ways of developing
software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

That is, while there is value in the items on
the right, we value the items on the left more.

Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

© 2001, the above authors
this declaration may be freely copied in any form,
but only in its entirety through this notice.

http://agilemanifesto.org/

92

Manifesto para o Desenvolvimento Ágil de Software.

Ao desenvolver e ao ajudar outros a desenvolver software,
temos vindo a descobrir melhores formas de o fazer.
Através deste processo começámos a valorizar:

Indivíduos e interacções mais do que processos e ferramentas
Software funcional mais do que documentação abrangente
Colaboração com o cliente mais do que negociação contratual
Responder à mudança mais do que seguir um plano

Ou seja, apesar de reconhecermos valor nos itens à direita,
valorizamos mais os itens à esquerda.

Kent Beck	James Grenning	Robert C. Martin
Mike Beedle	Jim Highsmith	Steve Mellor
Arie van Bennekum	Andrew Hunt	Ken Schwaber
Alistair Cockburn	Ron Jeffries	Jeff Sutherland
Ward Cunningham	Jon Kern	Dave Thomas
Martin Fowler	Brian Marick	

© 2001, os autores acima indicados
Esta declaração pode ser copiada e distribuída livremente, e de qualquer forma,
mas apenas se for feita de forma integral.

https://agilemanifesto.org/iso/ptpt/manifesto.html

93

4 valores presentes no manifesto

1) - Valorizar pessoas e interações, mais do que processos e ferramentas
   - Apesar dos processos e as ferramentas são importantes para o sucesso de um projecto
     - São as pessoas que as usam, que as implementam, logo a ênfase de ser nas pessoas e nas suas interacções
2) Valorizar o software a funcionar, mais do que a documentação completa
   - Apesar da documentação ser importante o foco deve estar em entregar ao cliente "valor real" (software a funcionar)
3) Valorizar a colaboração com o cliente, mais do que a negociação contratual
   - Tipicamente os clientes era vistos como elementos externos
     - A envolvência só ocorria no início e no fim do ciclo de desenvolvimento do produto.
   - O cliente passa a estar presente ao longo do ciclo de desenvolvimento
     - Trabalho conjunto entre a equipa e o cliente para desenvolvimento e melhoria do produto
4) Valorizar dar resposta à mudança, mais do que cumprir com o planeamento
   - Abordagem flexiva e adaptativa
     - Incorporação ao longo do desenvolvimento associado a ciclos de desenvolvimento de curta duração

94

94

Princípios do Manifesto Ágil

Seguimos os seguintes princípios:

A nossa maior prioridade é,
desde as primeiras etapas do projecto,
satisfazer o cliente através da entrega
rápida e contínua de software com valor.

Aceitar alterações de requisitos,
mesmo numa fase tardia do ciclo de desenvolvimento.
Os processos ágeis potenciam a mudança
em benefício da vantagem competitiva do cliente.

Fornecer frequentemente software funcional.
Os períodos de entrega devem ser de poucas semanas
a poucos meses, dando preferência a períodos mais curtos.

O cliente e a equipa de desenvolvimento
devem trabalhar juntos, diariamente,
durante o decorrer do projecto.

Desenvolver projectos com base em indivíduos motivados,
dando-lhes o ambiente e o apoio de que necessitam,
confiando que irão cumprir os objectivos.

O método mais eficiente e eficaz de passar
informação para e dentro de uma equipa de desenvolvimento
é através de conversa pessoal e directa.

A principal medida de progresso
é a entrega de software funcional.

Os processos ágeis promovem o desenvolvimento sustentável.
Os promotores, a equipa e os utilizadores
deverão ser capazes de manter,
indefinidamente, um ritmo constante.

A atenção permanente à excelência técnica
e um bom desenho da solução aumentam a agilidade.

Simplicidade – a arte de maximizar a quantidade de trabalho
que não é feito – é essencial.

As melhores arquitecturas, requisitos e desenhos
surgem de equipas auto-organizadas.

A equipa reflecte regularmente
sobre o modo de se tornar mais eficaz,
fazendo os ajustes e adaptações necessárias.

https://agilemanifesto.org/iso/ptpt/principles.html

95

95

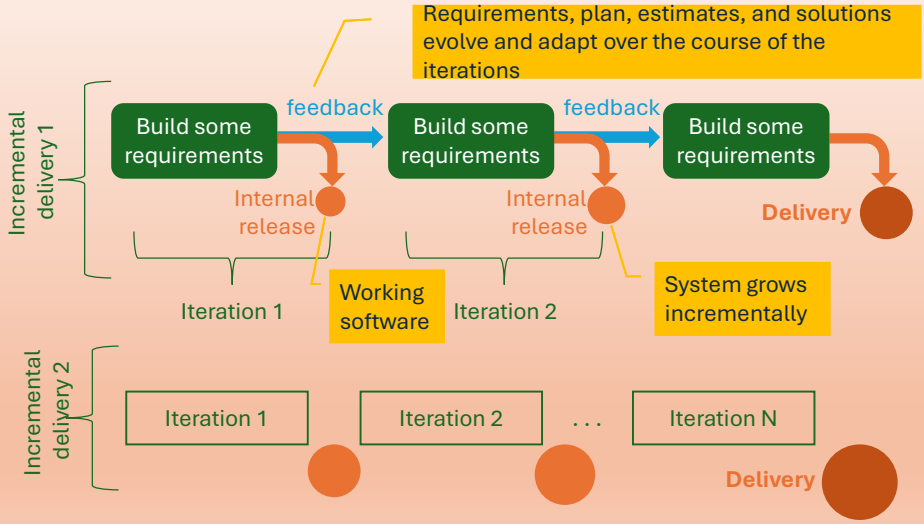
Os princípios do desenvolvimento Ágil

1. A nossa maior prioridade é, desde as primeiras etapas do projecto, satisfazer o cliente através **da entrega rápida e contínua** de software com valor.
2. **Aceitar alterações de requisitos**, mesmo numa fase tardia do ciclo de desenvolvimento. Os processos ágeis potenciam a mudança em benefício da vantagem competitiva do cliente.
3. **Fornecer frequentemente software funcional**. Os períodos de entrega devem ser de poucas semanas a poucos meses, dando preferência a períodos mais curtos.
4. **O cliente e a equipa** de desenvolvimento devem **trabalhar juntos**, diariamente, durante o decorrer do projecto.
5. Desenvolver projectos com base em **indivíduos motivados**, dando-lhes o ambiente e o apoio de que necessitam, confiando que irão cumprir os objectivos.
6. O método mais eficiente e eficaz de passar informação para e dentro de uma equipa de desenvolvimento é através **de conversa pessoal e directa**.
7. A principal **medida de progresso** é a entrega de **software funcional**.
8. Os processos ágeis promovem o **desenvolvimento sustentável**. Os promotores, a equipa e os utilizadores deverão ser capazes de manter, indefinidamente, um ritmo constante.
9. A atenção **permanente à excelência técnica e um bom desenho da solução** aumentam a agilidade.
10. **Simplicidade** – a arte de maximizar a quantidade de trabalho que não é feito – é essencial.
11. As melhores arquitecturas, requisitos e desenhos surgem de **equipas auto-organizadas**.
12. A **equipa reflecte** regularmente sobre o **modo de se tornar mais eficaz**, fazendo os ajustes e adaptações necessárias.

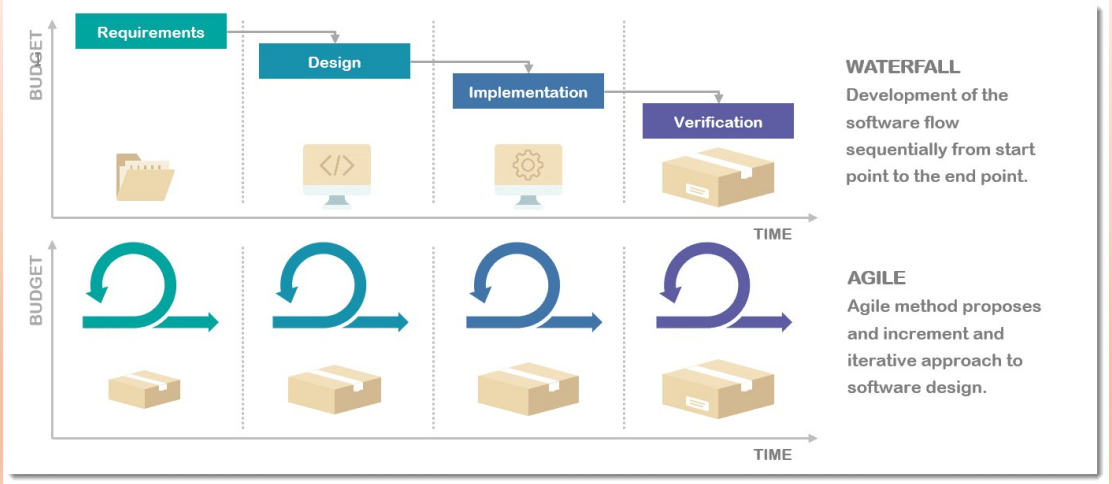
What makes a method Agile?

- Short iterations
- Incremental design
- User involvement
- Minimal documentation
- Informal communication
- Adaptive planning
- Evolutionary delivery

Iterative, evolutionary, adaptive and incremental delivery

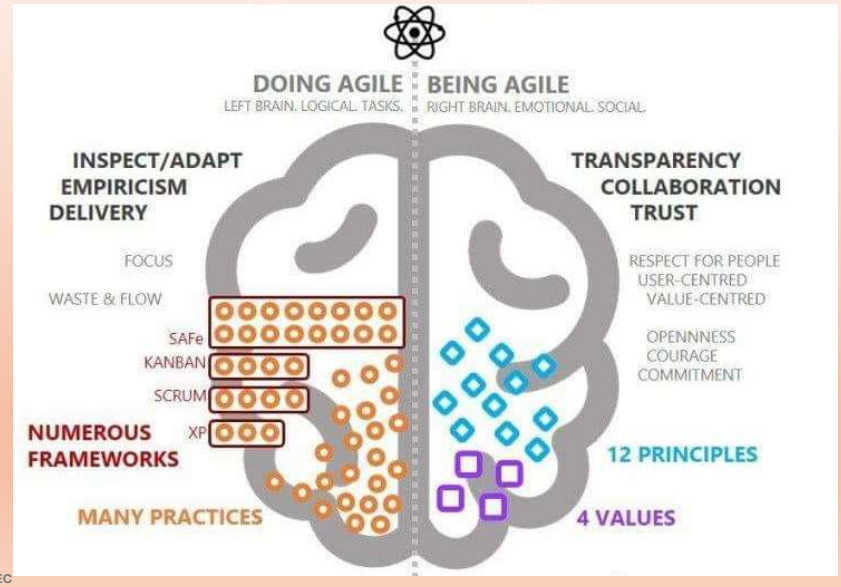


98



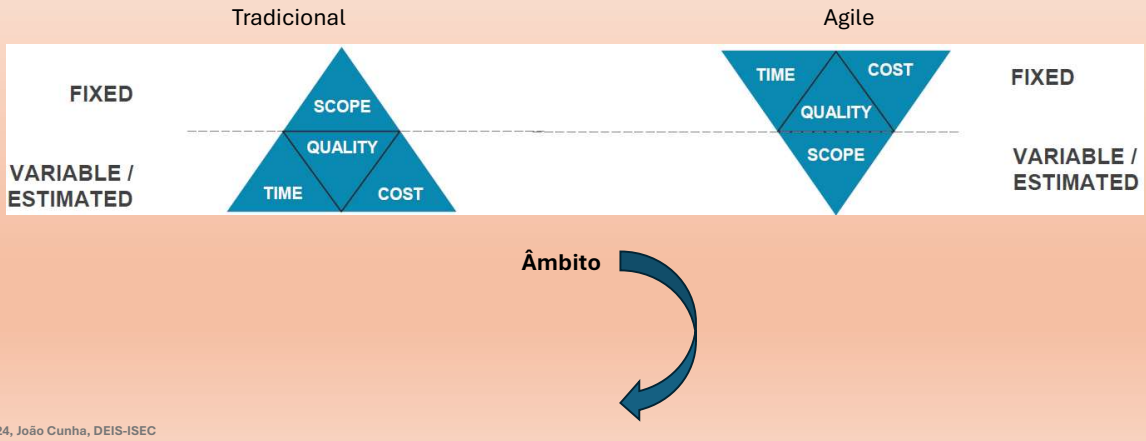
99

Agile Mindset



100

Motivation



101

Em resumo



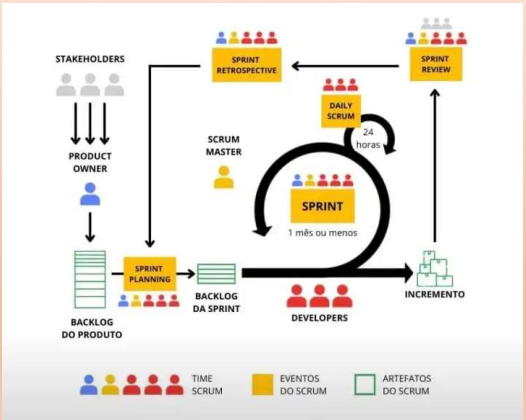
Difference between Agile and Waterfall
https://www.youtube.com/watch?v=5RocT_OdQcA

102

102



Scrum



103

103

Origem do Scrum

- 1986 - Hirotaka Takeuchi e Ikujiro Nonaka publicam na revista Harvard Business Review o artigo intitulado "The New New Product Development Game“
  - https://hbr.org/1986/01/the-new-new-product-development-game
  - Descreviam a forma como empresas como a Honda, a Canon ou a Fuji-Xerox conseguiam obter bons resultados no desenvolvimento de novos produtos com a utilização de abordagens escalonáveis e baseadas em trabalho em equipa. Equipas estas que eram autónomas e auto-organizadas.
  - Scrum é um termo “emprestado” do râguebi
    - Os autores fazem a comparação da metodologia ao “trabalho” de uma equipa de râguebi

104

104

https://hbr.org/1986/01/the-new-new-product-development-game

Under the old approach, a product development process moved like a relay race, with one group of functional specialists passing the baton to the next group. The project went sequentially from phase to phase: concept development, feasibility testing, product design, development process, pilot production, and final production. Under this method, functions were specialized and segmented: the marketing people examined customer needs and perceptions in developing product concepts; the R&D engineers selected the appropriate design; the production engineers put it into shape; and other functional specialists carried the baton at different stages of the race.

Under the rugby approach, the product development process emerges from the constant interaction of a hand-picked, multidisciplinary team whose members work together from start to finish. Rather than moving in defined, highly structured stages, the process is born out of the team members' interplay (see Exhibit 1). A group of engineers, for example, may start to design the product (phase three) before all the results of the feasibility tests (phase two) are in. Or the team may be forced to reconsider a decision as a result of later information. The team does not stop then, but engages in iterative experimentation. This goes on in even the latest phases of the development process.

105

105

3 pilares de suporte do Scrum

- Transparência
  - Todos devem saber a todo o momento o que se está a passar com o projecto
- Inspecção (validação)
  - Existência de meios de controlo ou verificação do trabalho realizado e o compare com o esperado
- Adaptação
  - Responder de forma rápida às mudanças
    - O cliente pode mudar de ideias
    - O mercado pode mudar
    - A legislação pode mudar
    - ....

SCRUM

- Scrum is an **iterative, incremental** method for managing the systems development.
- Emphasizes a set of project management **values and practices**.
- **Does not define any specific software development** technique for the implementation phase; it is a wrapper for existing engineering practices.
- Concentrates on **how the team members should function**, promoting self-directed teams and daily team measurement.
- Produces a **potentially shippable** set of functionalities at the end of every iteration.
- Has strong emphasis on **empirical** rather than defined process.

6 Princípios do Scrum

1. **Controlo empírico do processo**
   - Decisões efectuadas com base na evidências em vez de um planeamento inicial detalhado.
2. **Auto-organização**
   - Ao contrário do estilo de comando e controlo utilizado nas equipas "tradicionais" de desenvolvimento de produtos
3. **Colaboração**
   - Trabalho de equipa entre as partes interessadas e quem está no processo de desenvolvimento
4. **Priorização baseada em valor**
   - Entregar o maior valor no mais curto espaço de tempo, logo a priorização é efectuada com base neste princípio
5. **Timeboxing**
   - O tempo é tratado como um constrangimento
   - A cada processo ou actividade é atribuído um período fixo de tempo
6. **Desenvolvimento iterativo**
   - O cliente pode não ser capaz de definir todas as especificações do produto final
   - Vão surgindo alterações ao longo do desenvolvimento (por exemplo a pedido do cliente)
   - Flexibilidade para incorporar mudanças de requisitos a cada fase de desenvolvimento

108

108

Key practices

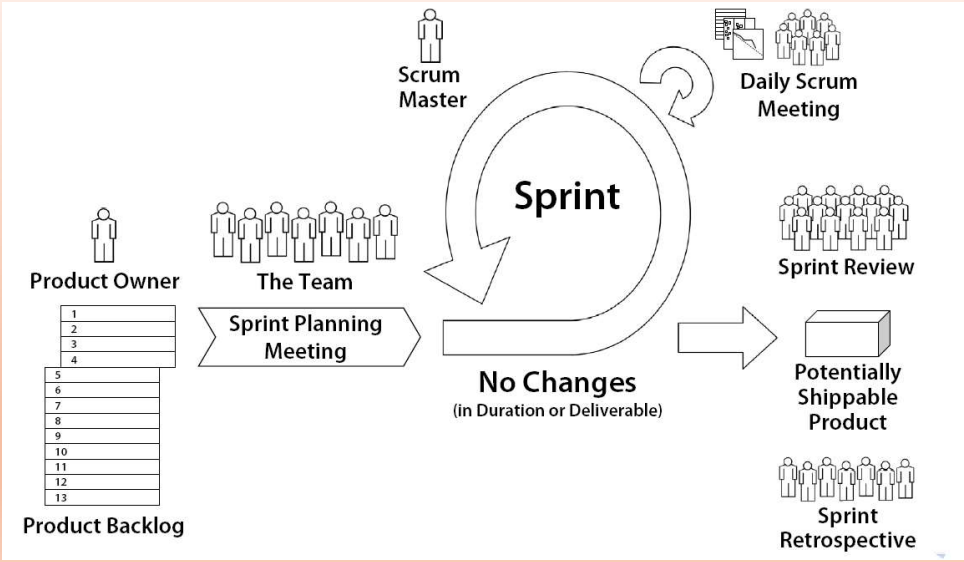
- self-managing team
- no external addition of work to an iteration, once chosen
- daily stand-up meeting
- usually 2 to 4-week iterations
- demo to external stakeholders at end of each iteration
- client-driven adaptive planning

[75] @GPS2324, João Cunha, DEIS-ISEC

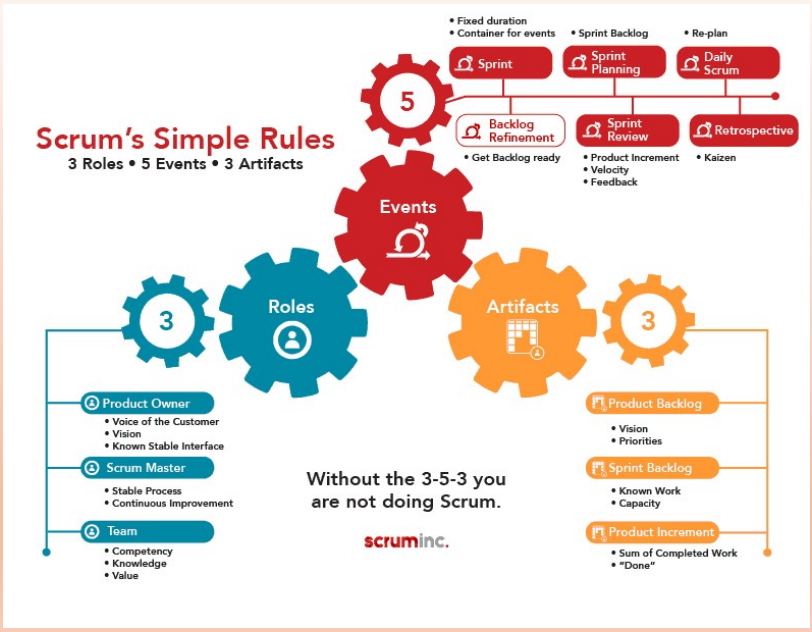
109

109

Scrum Process flow



110



111

Scrum framework

- Roles
  - Product Owner
  - Scrum Master
  - Team
- Artifacts
  - Product Backlog
  - Sprint Backlog
  - Product Increment
- Events
  - Sprint
  - Sprint Planning
  - Daily Scrum Meeting
  - Sprint Review
  - Sprint Retrospective
  - Backlog Refinement



Roles / Papeis

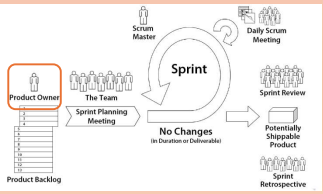
Scrum



Product Owner

- Owns the vision of what should be produced
- Gets inputs from customers, end-users, team, managers, executives, industry experts, etc.
- Is responsible for creating and prioritizing the Product Backlog
- Monitors the project against the stated goals
- Only one person

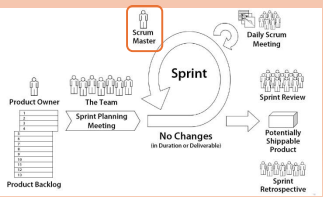




Scrum Master

- Responsible for setting the team up for success:
  - Establishes Scrum practices and rules
  - Shields the team from outside disturbances
  - Removes obstacles
- Represents management in the project
- Can be played by a team-member.
- Without a Scrum Master, the team has a high risk of failure.

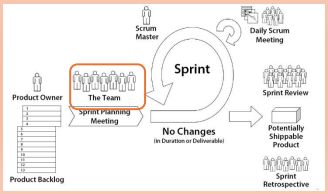




The Development Team

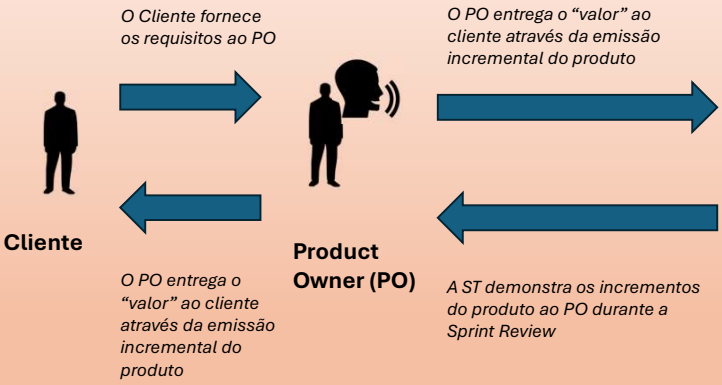
- Until 10 people
- Cross-functional with undifferentiated roles
  - It has all the skills to produce finished product – designers, coders, testers, etc.
  - Everyone contributes based on competency, rather than just job title
- Self-organizing and self-managing
- Responsible for making a commitment and managing itself to hit the goal
- Authority to do whatever is needed to meet the commitment



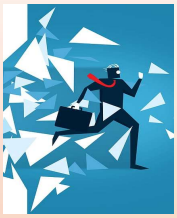


116

Papeis Centrais no Scrum - Resumo



Scrum Master (SM)



- *O SM assegura um correcto ambiente de trabalho para a ST*
- *"remove obstáculos"*



Scrum TEAM (ST)

117



User stories

Algumas orientações e regras

User stories (US)

- São declarações curtas, simples e fáceis de entender, que resultam/fomentam uma melhor comunicação entre o cliente e a equipa
- São uma forma simples de documentar, para o cliente, os requisitos e as funcionalidades
- Os requisitos do cliente são expressos nas US de maneira resumida, simples e fácil compreensão

Agile User Stories

- A user story is an informal, general explanation of a software feature **written from the perspective of the end user**. Its purpose is to articulate how a software feature will provide value to the customer.
- User story is a show description of a specific customer requirement:
  - **RGB** format: Role; goal; benefit
  - **As a** <type of user> , **I want** <some goal> **so that** <some reason>
- Type of user:
  - Who are we building this for?
  - Who is the persona ?
- Some goal:
  - What is the intention?
  - What are they realy trying to achieve?
- Some reason:
  - What's the overall benefit?
  - What is the big problem that needs to be solved?

Não é na nossa !!!.

120

INVEST – How a good user story should be

- Metodologia para facilitar a garantia da qualidade de uma US
- **I** - Independent (of all others)
  - ❑ cada US deve ser independente das restantes, de forma a evitar problemas de estimativa e priorização
- **N** - Negotiable (not a specific contract for features)
  - ❑ Cada US deve permitir oportunidades para negociar com o cliente, apresentando breves descrições, com pouco nível de detalhe
- **V** - Valuable (or vertical)
  - ❑ Cada US deve ter por base a entrega de valor ao cliente
- **E** - Estimable (to a good approximation)
  - ❑ Cada US deve permitir a quem vai desenvolver estimar com facilidade.
- **S** - Small (so as to fit within an iteration)
  - ❑ Deve representar um esforço reduzido, de forma a evitar erros de estimativa e definição do âmbito.
- **T** - Testable (in principle, even if there isn't a test for it yet)
  - ❑ Cada US deve ser testável. Algo que não se possa ser testado não deve ser desenvolvido.

121

US

- Uma US bem escrita consegue revelar 3 coisas sobre os requisitos
  - O quê (What?)
  - Quando (When?)
  - Quem (Who?)

  } **3 W**

- Os requisitos do cliente são expressos nas US de maneira resumida, simples e fécil compreensão.

Three C's - Components of a User Story

- To write a good user story:
  - **Card** – a physycal token, with Title, Role-Goal-Benefit
  - **Conversation** - taking place at a different time, between customers, users, developers, that may include documentation
  - **Confirmation** – the acceptance criteria that should be fulfield and tested to ensure the user requirements have been met and delivered correctly

 ©2008-2023 João Cunha / Ricardo Filipe, DEIS-ISEC

Acceptance Criteria

- Allows to check if the user story is really "done".
- Gathers the requirements and all scenarios that allows to understand that the user story is fully finished.
- Used to:
  - Define scenarios
  - Achieve consensos
  - Serve as a base for testing
  - Manage expectations

[98] @GPS2324, João Cunha, DEIS-ISEC

Critérios de aceitação

- Os critérios de aceitação garantem objectividade da US, uma vez que muitas das vezes as US tendem para a subjectividade, para que sejam consideradas "feitas" ou "não"
- Os critérios de aceitação permitem à ST clarificar o que é esperado de determinada US, evitam ambiguidades e ajudam a alinhar expectativas

Agile User Stories and Acceptance Criteria examples

Add Prospect

As a property manager I want to add a new prospect to the lead management system so I can track my interactions with the prospect.

Conditions of Satisfaction

Capture name, email, phone #, contact date, contact format, lease type, and move-in date

Verify prospect is associated with an existing campaign

DESCRIPTION:

As a customer, I want to order and pay for the book via a secure web-based form, so that my credit card information is safe.

ACCEPTANCE CRITERIA:

- All mandatory fields must be completed before a customer can submit a form.
- Information from the form is stored in the customer orders database.
- Payment can be made via Amex, Master Card, or Visa credit card.
- The system shall accurately calculate and apply sales tax.
- The system shall accurately calculate and apply shipping charges.
- The customer shall be able to verify the accuracy of the order.
- An acknowledgment email is sent to the customer submitting the form.
- Protection against spam is working.
- The code should be deployed and running in Staging environment

Given/When/Then Acceptance Criteria

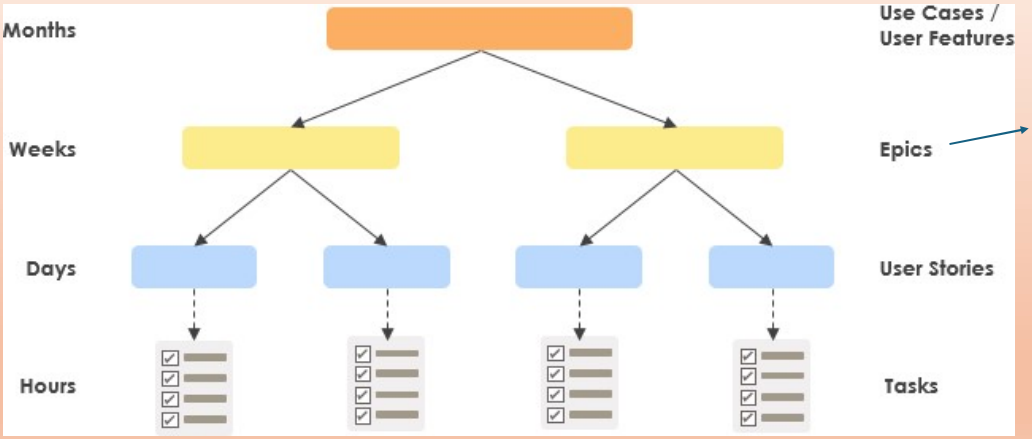
- **Scenario:** (name of the behavior)
  - **Given** a context
  - **When** an event happens
  - **Then** there is an outcome
  - **And** (may be used to continue any of the 3 previous statements)

GWT examples

User story: As a user, I want to be able to request the cash from my account at an ATM so that I will be able to receive the money from my account quickly and in different places.	
Scenario 2	Requesting the cash from an overdrawn account
Given	The account is overdrawn
And	The card is valid
When	The customer requests the cash
Then	Ensure the rejection message is displayed
And	Ensure cash isn't dispensed

User story: As a user, I want to be able to recover the password to my account, so that I will be able to access my account in case I forgot the password.	
Scenario	Forgot password
Given	The user navigates to the login page
When	The user selects <*forgot password*> option
And	Enters a valid email to receive a link for password recovery
Then	The system sends the link to the entered email
Given	The user receives the link via the email
When	The user navigates through the link received in the email
Then	The system enables the user to set a new password

Use Cases, Epics, User Stories, Tasks



- US de alto nível, escritas numa fase inicial do projecto, quando os requisitos ainda são definidos de forma mais geral

Further Reading

- How to write a Good User Story https://www.youtube.com/watch?v=P3hbbKnzm4A
- https://www.atlassian.com/agile/project-management/user-stories
- https://www.agilealliance.org/glossary/user-stories/
- Alexsoft, Acceptance Criteria for User Stories: Purposes, Formats, Examples, and Best Practices, 18/5/2021 (https://www.altexsoft.com/blog/business/acceptance-criteria-purposes-formats-and-best-practices/)

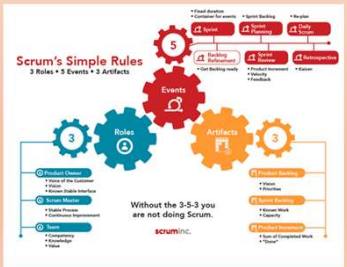
©2011-2023 João Cunha, DEIS-ISEC

130



Artifacts

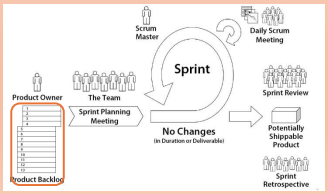
Scrum



131

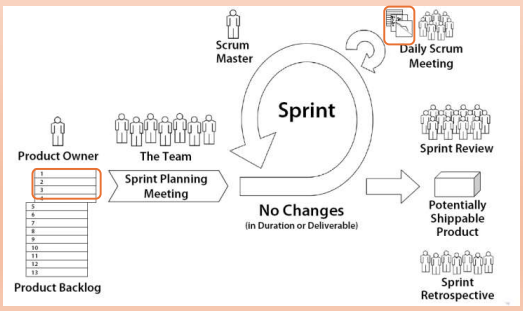
Product Backlog

- Prioritized list of work to be performed on a product
  - Requirements oriented or based on user stories (US)
- Anyone can contribute to backlog items
  - Product Owner is responsible
- Product Owner is responsible for prioritization
  - Focus on business value (80% of the value comes from 20% of the features)
  - Reviewed at the start of each sprint or during Backlog Refinement



Sprint Backlog

- Lists of the tasks to be executed during the sprint
  - Selected during the Sprint Planning
- Status of the tasks updated daily



- Cada membro da equipa selecciona as tarefas nas quais prevê trabalhar
- Como a ST é auto-organizada, a decisão de quem vai fazer o quê é dos próprios membros da equipa.
- Nem o PO nem SM dão indicações à equipa sobre quais os itens a trabalhar ou sobre como trabalhar.


isec
Politécnico de Coimbra
Gestão de Projecto de Software
Licenciatura em Engenharia Informática
2024/2025
SCRUM (2)
A4
134